# 一章

1. 码值转换

① 二→十：权值展开

② 十→二

1. 除2取余法（整数）
   - ① 书写左边算到1
   - ② 读数顺序↑
2. 乘2取整法（小数）
   - ① 读数顺序↓

③ 二⟷十六

- ① 整数部分可由小数点开始向左每4位进行分组。
- ② 小数部分可由小数点开始向右每4位进行分组

④ 十⟷十六

2. 编码

① 真值：有"+、-"号

↳ 机器数：将真值中的"+、-"变成0和1

② 原码：正数符号位用0表示，负——1——，数值部分随后表示。

③ 反码：正数：原码、反码、补码相同。

负数：原码符号位不变，数值位按位取反

④ 补码：① 负数的反码+1

② 补码=补码=原码

↳ 任何符号数都是以补码形式存储的

↳ 所有负数在计算机中都是以补码形式存放的

↳ 8位补码：-128～+127

无符号数：0～255

16 -32768～+32767

0～65535

1. 二—十进制编码（BCD）

① 用4位二进制代码表示1位十进制数

（0000B～1001B）（0D～9D）

② 进行BCD码加法时（二进制加法）

若① 低4位之和大于9 或

② 低4位向高4位有进位时，

低4位要进行加6修正。

4. ASCII码：

0～9：30H～39H

A：65D

a：97D

) space 32H

利用7位二进制代码来表示字符，再加上一位校验位，故计算机用1个字节来表示1个字符。

~~BCD与ASCII的转化：~~

~~ASCII始终比BCD大48D，即30H~~

十六进制数与ASCII编码之间的关系：

① 十六进制数0～9的ASCII为30H～39H，差值为30H

② 十六进制数A～F的ASCII为41H～46H，差值为37H

（41H－0AH=37H）

在进行补码进行运算时，

可将减法转换为加法。

2. 计算机的硬件组成.

- cpu
  - 通用寄存器
  - 运算器 ① 计算机基本组成部分
  - 控制器 ④
- 存储器 ② → = 进制数的每一位就是数据的最小存储单位 (bit)
  - RAM
  - ROM

  字节 (Byte) — 基本数据单位。

  1B = 8b;

  $2^{10}B = 1K$
- I/O ③
- 总线
  - 系统总线 { AB, CB, DB } ⇄ I/O ⇄ { D, C, S (状态线) } — cpu ~ 外围设备.
  - 外部总线

3. 语言.

- 机器 — cpu唯一直接识别.
- 汇编
- 高级

算法

- 准则 P17
- 步骤
  - 自顶向下
  - 模块化设计
    - ① 小问题
    - ② 只有顺、选、循环
    - ③ 独立
    - ④ 上层调用

4. 单片机 — 单片微控制器. P15

5. 嵌入式 P17.

6. 系统组成

- 硬件系统
  - 最小系统
  - 最小功耗系统
  - 典型系统
- 软件系统
  - 系统软件 { 监控程序, 操作系统 }
  - 应用软件

P17. (简答题)

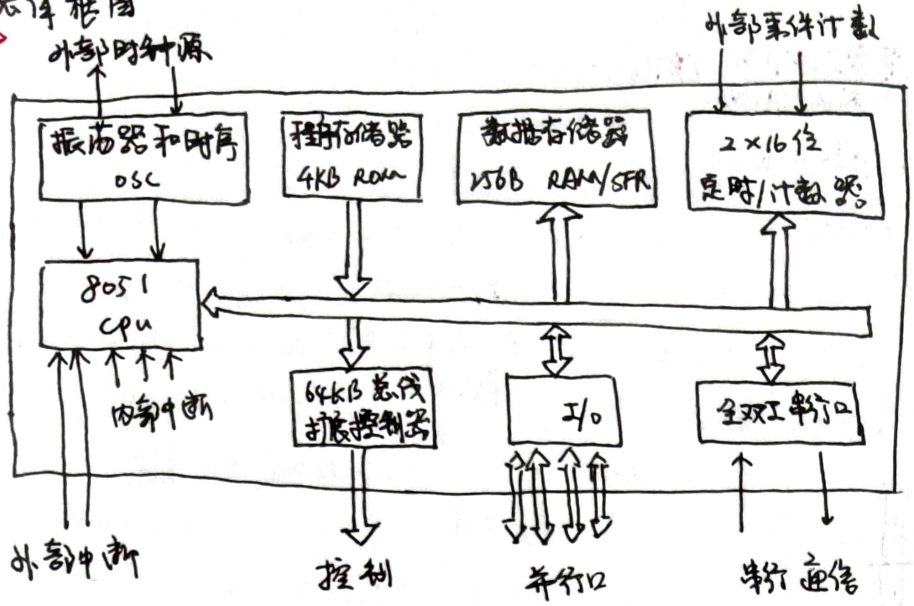
几种单片机内部资源和区别.

- ✓ 8031 — 无ROM
- ✓ 8051 — 4KB 掩膜ROM
- ✓ 8751 — 4KB 可编程EPROM
- 89C51 — 4KB 闪速EEPROM
- 89S51 — 4KB FLASH闪速程序存储器

8052、8752、89C52 — 8KB 掩膜ROM

二. 硬件结构

1. 总体框图




2. 各部件

1. CPU

① 控制器

① 程序计数器 PC

① 16位

② 用来存放CPU要执行的下一条指令存储单元的地址。(作用) 存放在程序存储器中

③ 特点

① 自动加1

② 跳转指令时会被修改

③ 3程序调用或中断：

i) PC当前值保存→压栈 (只有直接地址) push direct SP←(SP)+1 ; (SP)←(direct) 先+1再压栈。

pop direct ; (direct)←(SP), ; SP←(SP)-1 先为找再减1

ii) 3程序入口地址或中断矢量送入PC.

④ 复位：PC内容为0000H

② 指令寄存器 IR、指令译码器及控制逻辑电路.

② 运算器 (作用：算术运算、逻辑运算)

① ALU (算术逻辑运算部件)

② ACC (累加器)

① 8位.

② 助记符 A.

③ 作用 ① ACC是ALU输入之一，且ALU运算结果一般送回ACC。

② 大多数指令与ALU有关

③ 存放 使用次数较高的操作数和中间结果

③ PSW (程序状态字) 地址：D0H (不是00是D0)

| D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|
| CY | AC | F0 | RS1 | RS0 | OV | F1 | P |

对ADD指令，位6和位7同时有进位或同时无进位→OV=0

否则 OV=1

补充.

CY: ① ALU进位/借位标志位.

② 位累加器C.

RS1、RS0：寄存器组选择位

注：任一时刻，只能有一个八寄存器激活.

OV：溢出标志位

P：奇偶校验位.

偶数个1：P=0

奇 P=1

2. RAM

3. ROM

4. 定时/计数器

①定时器：固定时间间隔的内部机器周期计数。

②计数器：外部事件所产生的脉冲进行计数。

（晶体振荡器提供输出给定时器）

5. 中断系统

5个中断源 {3个由 T0 T1 和串口产生。 / 2个外 INT0 INT1

6. 并行 I/O. 64个位：与外部存储器或I/O交换信息

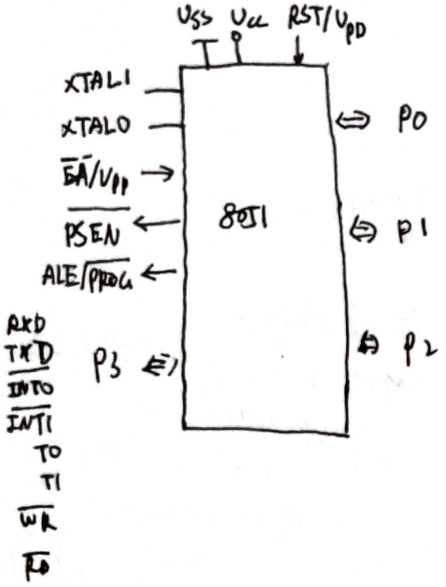
P0. P2 作数据地址口，P0作数据/地址分时复用

7. 串行口 全双工

对数据各位按序一位一位地传输

8. 时钟电路 OSC：产生单片机最基本的时间单位

3. 引脚 (40个) 逻时针排列




①主电源引脚

Vcc. +5V

Vss. 0V

②. 电种电路引脚

两种方法：①自激振荡器 {XTAL1 输入 / XTAL2 输出

②接外部信号源 {XTAL1 接地 / XTAL2 外部信号源

7、读缓存器 ANL P0,#0FFH

读引脚 MOV A,P0

③ 控制信号

① RST/VPD

RST：复位

VPD：备用电源

② ALE/PROG

ALE：低8位地址锁存

外部存储器的16位地址信号由P0输出低8位. P2输出高8位，ALE在输出为脉冲下降沿时16位地址低8位分别锁存

$f_{ALE}=\frac{1}{6}f_{osc}$

③ PSEN—OE 读外部ROM时，低电平有效

读选通信号，实现外部ROM的读操作.

每个机器周期2次有效.

④ EA/VPP

EA：外部程序存储器允许.

● EA=1 时，从内部ROM开始读取.

超过0FFFH时，自动转向外部1000H；

● EA=0 时，只访问外部.

0FFFH

3×4个1

$2^{12}B=2^2\times2^{10}B=4KB$

P2.7～P2.4.

⑤ 并行口 P0～P3

输入前要先写高电平

（其他都是准双向）

0，1，高阻态

P0：8位，双向，三态（内部不带上拉电阻）

外接1kΩ

其余P口带

① 数据地址分时复用

② 以吸收电流方式驱动8个LS型TTL负载

（P1 P2 P3 都是4个）

P1：

P2：

① 访问外部存储器时，输出高8位地址

P3：① 第二功能

补充：

① 掌握 P0 → 数据地址分时复用

P2 通常作为数据地址口

② 由于①的原因，P0. P2 和部分的原因导致的

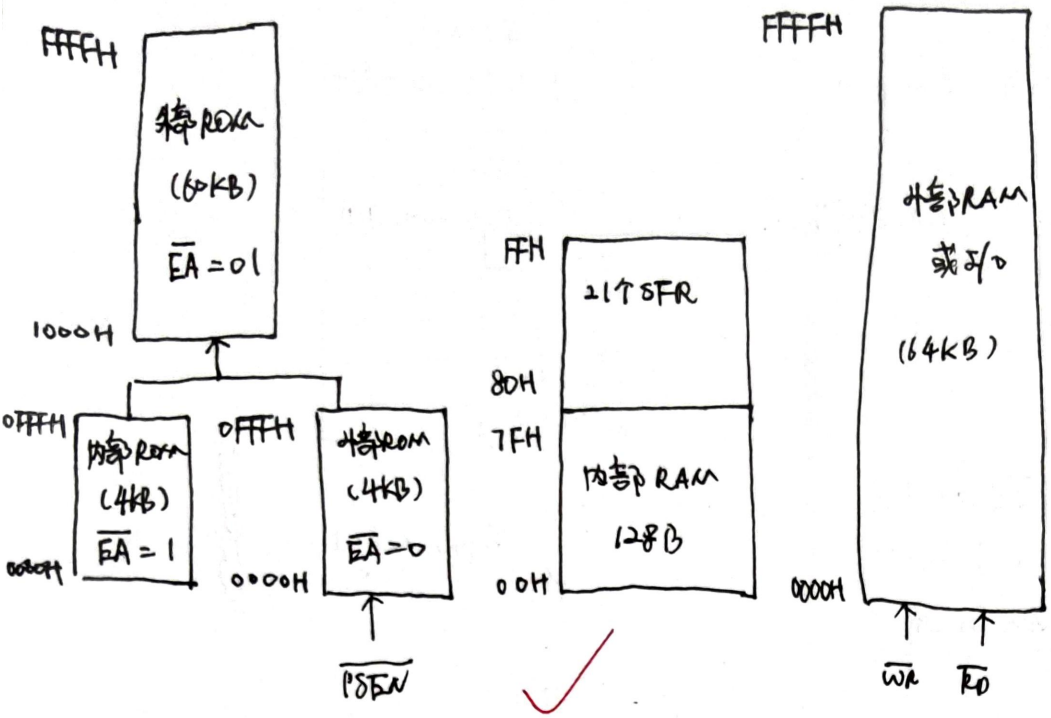
P3.6. P3.7 不能用于做通用I/O口.

已用 WR 和 RD

其余并行口可随意用于I/O（P1. P3.0～P3.5）

③ 51的I/O要规划使用，一个管脚不能已定做输入又作为输出口，做输入就只进行读操作，而不要对其进行写（输出同理）

# 4. 存储结构




## 1. 程序存储器 (非易失性)

① 作用：存放已编制好的程序及程序中用到的常数.

② 内内、内外地址是连续的，即内部ROM读完自动跳下一地址的外部ROM

③ 中断入口地址

⇒ 中断处理子程序第一条代码存放到中断入口地址处，

第二条指令需是跳转指令，跳到主程序的存储空间.

补充：启动地址：0000H

复位地址：0000H

⇒ 第1条指令要放到0000H，

接着用跳转指令转移到0023H以后的空间. (空出中断入口地址)

## 2. 数据存储器.

1. 通用寄存器 4组. R0~R7.

① 作用：暂存中间结果，~~寻址方式中不可缺少~~

② 特点：任一时刻 CPU只能选用一组工作寄存器，其余三组可作为一般数据存储器

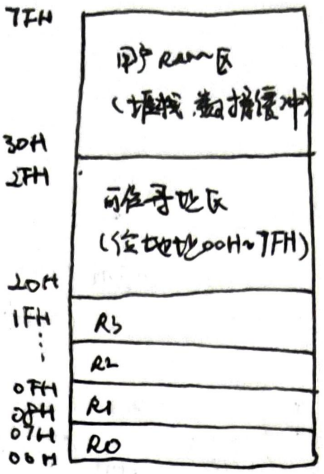
③ CPU复位后，自动选择第0组

2. 可位寻址区 20H ~ 2FH (128)

① 既可按字节，又可按位

② 字节地址单元和位地址单元表示相同，但含义不同。

如何区分？根据指令区分（或根据操作数）




3. 只能字节寻址RAM (30H ~ 7FH)

堆栈：①保护断点，保护现场

②因为SP复位以后内容为07H，这会占用R1寄存器，因此应初始化SP：使其值于内部RAM高端。

例 MOV SP, #70H

③ SP → 栈顶。

先进后出，向上生长

4. SFR (离散分布于内部RAM高128B) 对空闲地址操作是无意义的。

(放CPU的状态（状态字、控制字))

① 只能采用直接寻址

② 字节地址末位是0H或8H 才能进行位寻址操作

③ 特殊 ACC, B, PSW, SP, P0~P3

SP不能位寻址

课本P34，表2-4

内部RAM低128B P3.2 P3.3

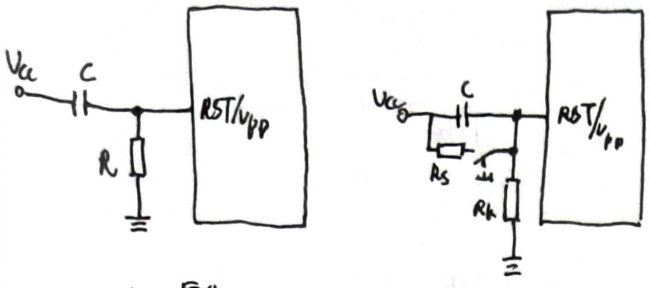
综2、4得，位地址空间211个。(128 + 83)

内部RAM 20H~2FH

存于SFR

5. 工作方式

① 复位方式 { 上电复位 / 按键复位 { 按键脉冲复位 / 按键电平复位 }




上电复位　　按键电平复位.

① $U_{RST} = U_R = V_{CC} - U_{电容C}$

② 如何复位？　计数器：1个机器周期

在RST上加上持续2个机器周期以上的高电平

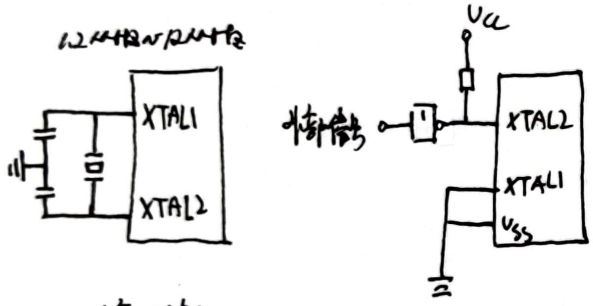
③ 在复位有效期间，ALE、$\overline{PSEN}$ 脚均为高电平.

内部RAM不受复位影响.

④

| | |
|---|---|
| PC | 0000H |
| SP | 07H |
| DPTR | 0000H |
| P0~P3 | 0FFH ✓ 为0V则为低电平 |

⑤ 时钟




内部时钟　　外部时钟

① 控制器作用：单片机的时序是指CPU执行指令时所需控制信号的时间顺序，用于内外存储器和I/O口的控制

② 名词

1. 时钟周期（振荡周期）：$T = \frac{1}{f_{osc}}$

① 单片机操作时序中的时间单位.

② 时间脉冲是计算机的基本工作脉冲，控制计算机工作节奏

2. 机器周期 = 12个时钟周期

① 单片机基本操作周期.

② 1个机器周期 = 6个状态周期

1个状态 = 2个节拍

1个节拍 = 1个时钟周期

⑥ 程序执行方式是单片机最基本的工作方式

1. 连续执行

2. 单步执行目的：调试方式；原理：利用外部中断0

特殊功能寄存器PCON（电源控制寄存器）

| PCON.1 | PCON.0 |
|---|---|
| PD | IDL |

PD=1，掉电　IDL=1，空闲

⑦ 节电方式 HMOS：

1. 掉电方式（当$V_{CC} < V_{PD}$）

MOV PCON, #02H　同时=1，先进入掉电

STR初始化，内部RAM保持不变 ⇒ 利用INT0通知CPU，CPU执行中断服务程序，将有关数据保存到内部RAM

⑧ 2. 空闲方式

MOV PCON, #01H

① CPU停止工作，但中断、串行口、定时/计数器继续工作

② 通用寄存器保持原有状态，时钟电路继续工作.

③ ALE和$\overline{PSEN}$为高电平，无脉冲输出，处于无效状态.

退出方式：中断、复位.

I　II

3. 指令周期

① 执行一条指令所需要的时间

② 单周期指令、双周期 …、四周期指令

总线51单片机 12MHz时.　1μs = 1百万

| | |
|---|---|
| 时钟周期 | 1/12 μs |
| ✓机器 | 1 μs |
| 状态 | 1/6 μs |
| 指令 | 1~4 μs |

{ ① 取指令所需 / ② 执行指令所需 }

4. 单片机取指/执行时序

$$f_{ALE} = \frac{f_{osc}}{6}$$

① ALE信号：每个机器周期出现2次正脉冲.

作用① 允许随存储器地址

② 当作是否工作的标志信号，判断时用示波器测量

③ ALE信号每出现1次，CPU就进行一次取指令操作.

④ 4种取指令操作的时序. P41 图2-9

③ 访问外部RAM的时序

1. 两种情况：读操作、写操作

$\overline{RD}$　　$\overline{WR}$

2. 外部RAM时序图 P42. 图2-10.

注意：① 在读外部RAM期间，第一次的ALE信号无效，$\overline{PSEN}$ 也处于无效状态。

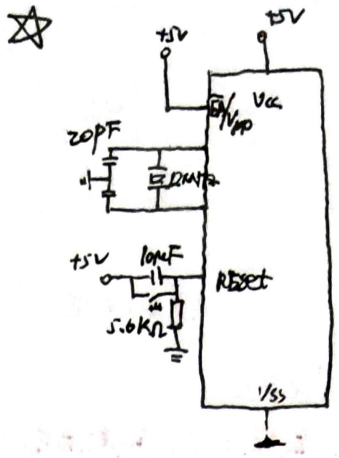
② 通常情况下，每一个机器周期中ALE信号两次有效，仅仅在访问外部RAM期间（MOVX）第二个机器周期才不发出第一个ALE脉冲

④ 访问外部ROM时序

注意：① ALE信号在8IP2有效时，$\overline{PSEN}$ 继续保持高电平或从低电平变为高电平无效状态。

5. 单片机最小系统

电源电路、电钟电路 复位电路



# 程序设计

## 1. 循环

① 初始化
循环体
循环控制 { 计数控制法 / 条件控制法

② 嵌套时应注意
① 允许外重嵌套内重 ✓
② 循环体不能交叉 ✗
③ 不能从循环程序外部跳到循环内部 ✗

## 2. 软件延时

① 12MHz
一个机器周期 1μs
DJNZ 2μs

② 软件延时不允许有中断

## 3. 代码转换程序

① BCD 码
压缩BCD码

② ASCII 码始终比 BCD 码大 48D
即 30H

## 4. 查表程序

① 根据自变量x，在表格中寻找 y=f(x)
② 发出读程序存储器 PSEN
③ 2条指令
MOVC A,@A+DPTR
MOVC A,@A+PC

④ 活动表格
MOV DPTR,#SQTAB
⋮
SJMP $ ⟶ 此程序一直执行而进入死循环
① 从头开始循环
② 原地踏步
SQTAB: DB

## 5. 程序运算

6. 冒泡排序

① n个数进行n轮，1轮进行(n-1)次

可以得到1个最大/小数

② 如果没有发生数据交换，则判断已经排序完成

③ 进行比较时用 CJNE 较方便

然后判断C于位。

相比于SUBB的优点：CJNE不改变第1和第2操作数中的内容，只影响C。

$f_{osc}$ = 12MHz；

延时1ms程序：

| | | | μs |
|---|---|---|---|
| DELAY: | MOV | R7, #0FFH | 1 |
| Loop: | NOP | | 1 |
| | NOP | | 1 |
| | DJNZ | R7, Loop | 2 |
| | RET | | 2 |

(1+4×255+1)=1023μs

≈1ms

汇编程序设计

①简单程序

②分支程序 → 条件转移类指令实现

③循环程序 → 减到必要的流程图

- i)大大缩短程序所占用的存储单元，但执行时间不会减少。
- ii)组成部分
  - ①初始化
  - ②循环体
  - ③循环控制 { 计数控制法 / 条件控制法
- iii)注意事项
  - ①允许外重嵌套内重
  - ②循环体不能交叉
  - ③不能从循环程序外部跳入程序内部。

④延时程序设计

- i) 12MHz → 1个机器周期 = 1μs
  指令 → 1μs ~ 4μs
- ii)用软件实现延时时，不允许有中断，否则会严重影响定时准确性
- iii)计算延时时长时由内层循环向外层循环计算。

⑤代码转换程序设计

⑥查表程序设计 看例3-8P.

i)注意：①根据自变量x，在表格中寻找 y=f(x)

②当共读程序存储器 PSEN

③2条指令：

MOVC A,@A+DPTR

MOVC A,@A+PC.

ii)使用静态表格，表格放在程序最后，RET前

⑦运算程序设计

例3-10中多字节加法 → ADDC.

⑧冒泡排序 例3-5I (每一轮都可得到1个最大的数)

i)原理：n个数进行n轮，1轮进行(n-1)次，就可得到1个最大的数，(升序)

如果没有发生数据交换，则判断排序已经完成

ii)基本算法(P84)

从起始地址开始，将相邻两数依次进行比较，如果大小次序与要求一致，则不变；否则交换次序，直至将最大示数放到最后(1轮)。再将n-1个数(前)继续比较。

iii)可采用双重循环实现冒泡排序.

外循环控制比较次数

内循环实现依次比较，交换数据.

⑨输入/输出程序

i)用高电平初始化I/O口.

ii)"每当P0.0由高电平变为低电平时，由P1口读入1个数据"要点得将P0.0先置成高电平，否则P0.0由高变低，P1口读完数据后P1口将一直读，无法输入.

⑩数字滤波程序设计

课本例：限幅滤波

# 第二章 指令

1. 指令格式（四分段格式）

[标号:] 操作码 [目的操作数][,源操作数] [;注释]

标号：指令地址符号，唯一不能缺省部分

参加操作的数据或数据地址

注意事项说明：

1. direct：8位内部RAM地址
2. rel：补码形式的8位常数，为偏移量，-128~+127
3. DPTR：数据指针

2. 寻址方式

① 立即寻址 (#)

注意：① #data

② 若立即数以首位为A~F，前面要加零

② 直接寻址

① SFR（寻址唯一方式）

② 内部RAM低128B

③ 寄存器间接寻址 (@) 存放操作数的地址

① 外部RAM（寻址唯一方式） { 256B字节 MOVX A,@Ri ; 64KB MOVX A,@DPTR

② 内部RAM 寄存器 R0、R1 256B ; DPTR 64K

④ 寄存器寻址

Rn、A、B、C、DPTR

注意区别ACC和A的寻址方式：

直接寻址 间接寻址

⑤ 基址变址寻址

① 只有3条

✓ MOVC A,@A+DPTR

MOVC A,@A+PC (PC+1)

JMP @A+DPTR

② 基址 DPTR、PC

变址（偏移量） A

③ 作用：访问固化在程序存储器中

⑥ 相对寻址（转移类指令）

① rel 8位有符号数

→即有正负，可以上下跳转 [-128B, PC, 127B]

= 当前PC值

② 目的地址 = 转移指令所在地址 + 转移指令字节数 + rel

⑦ 位寻址 (bit)

当对某个端口进行位寻址时，必须使用ACC.

(如ACC.7)

3. 指令系统

1. 数据传送类（不影响标志位）

1. 数据传送.　（A 影响奇偶标志位 P）

① MOV　（片内）

② MOVX　（片外）

MOVX　A, @Ri

MOVX　A, @DPTR

MOVX　@Ri, A

MOVX　@DPTR, A.

$\overline{WR}$. $\overline{RD}$ 有效

注：① 若 MOVX 中出现 @Ri，则在 MOVX 之前要先给 P2 赋值地址从 P0 输出

② DPTR 低8位从 P0 输出，高8位地址 P2 输出

③ 片外 RAM 的数据只能（MOVX）和 A 之间进行传送 !!

③ MOVC

MOVC　A, @A+PC　当前指令下面 256B（A 无符号）　(PC+1)

✓ MOVC　A, @ A+DPTR　64K

$\overline{PSEN}$ 有效

2. 数据交换　ASX中的内容交换

① XCH　A, X

② XCHD　A, @Ri

③ SWAP　A

第1操作数只能是A

A 与 (Ri) 的低4位交换

A 的高低4位交换

3. 堆栈操作指令

① PUSH　; SP先+1再压栈

② POP　; 先弹为再 SP-1

push direct

注：① 直接寻址

② 要初始化 SP，

eg 因为 SP 复位后是 07H，则 SP 为第一个数在 08H 的位置，占用了第1组寄存器.

③ 先进后出，向上生长

∴ 初始化 SP 的时候，要把 SP 为在较高的位置.

例如 MOV　SP, #70H

2. 算术运算类

1. 加法

目的 ← 目的操作数 + 源

① 不带进位加法
ADD.

② 带进位加法
ADDC (常用于多字节加法运算)
→ 目的 ← 目的 + 源 + CY.

第1操作数只能是 A

③ 加1指令
INC.

INC:
① 不影响PSW中任何标志位 (OV. CY. AC)
② 若原变量中内容为FFH, 则程序执行后为00H

④ 十进制调整指令
DA A
↓
只能用在加法指令之后, (ADD, ADDC, INC)之后,
影响CY, 不影响OV.

计算方法见P60, 例3-24.

ADD A, R3 A=56H; R3=67H
DA A

56
+67
123

结果: A=23H
CY=1

ADD、ADDC:

注: 各标志位变化情况. (CY. AC. OV)

① 位6和位7 同时进位 OV=0
(或同时没有进位)

不同时 (即一个进位 OV=1
另一个没进位)

② 如果位7有进位, CY=1

③ 位3 AC=1

④ 奇偶校验位P随时反应A中1的个数.

两个数相加只用ADD就可以.
三个数相加第1次可用ADD
2 ADDC

2. 减法

① 带借位减法 对PSW的影响同ADDC一样
SUBB

② 减1
DEC

① 对PSW的影响同INC一样
② 00H-1 = FFH
③ 和INC不同的是没有DPTR指令

2. 乘法

MUL AB A×B → B 高字节 → 若乘积>255, 则B中非0, OV=1
A 低字节 CY总为0.

3. 除法

DIV AB A/B → A (商)
→ B (余数)

若除数为0, OV=1

3, 逻辑运算

1. 单操作数 → 对字节时只能对A，其他的清0时，MOV R0, #00H

① 清0，CLR A
② 求反，CPL A
③ 循环：
  ① 左移 RL A / RLC A
  ② 右移 RR A / RRC A

} 都是对A进行操作

作用：

对 RL、RR：
① i) 当A的最高位为0时，RL即对(A)×2
  ii) 当A的最低位为0时，RR即对(A)÷2
② 跑马灯

对 RLC、RRL：
①判断第n位是1

2. 双字节

与等于运算不同点是，结果存放于第1操作数，而不一定是A。

① 与
ANL
  ① 按位与，对1为1 → 作用：①常用ANL指令屏蔽操作数中的某些位 ②判断有符号数运算结果的最高位，进而判断正负。
  ② 相与，屏蔽
  1，本位
② 或
ORL
  ② 按位或，对0为0 → ①可用于拆字(P78，例3-40)
  0，本位
  1，1
③ 异或
XRL
  ③ 相同为0，不同出1 → 让哪一位变负就与1异或
  0，本位
  1，求反

别忘记写分号

如让内部RAM 50H单元中高2位取反：
XRL 50H, #0C0H

## 4. 控制转移指令

1. 无条件转移 (2字节指令) PC+2

① 长跳 LJMP ;pc+2 ;PC←16位地址 16位, 64K

PC←addr15~0, PC15~11不变
PC当前值(加2后的值)的高5位
与指令的11位地址拼在一起
共同形成16位目标给PC

其中 addr16.
addr11
rel
常用标号表示
隐含地址信息

② 绝对转移(中跳) AJMP ;pc+2 11位, 2K
;pc 拼成新地址

$2^{11} = 2 \times 2^{10}$
$= 2K$

③ 短跳 SJMP rel PC←PC+2+rel 上下 -128B ~ +127
可用原地踏步 SJMP $ 256B
防止程序一直执行跑进入未知空间

④ 间接长转移指令 JMP @A+DPTR
- 根据A内容不同, 实现程序三路分支转移
- 64K.
- 不影响 A、DPTR内容, 也不影响标志位.

好 调用、转移类

2. 条件转移
- 状态无判断 (JB, JNB, JNC)
- LCALL
- (除 CJNE 为PC+3, 其余都是2)

① 累加器判零 JZ rel
(不影响A) JNZ rel

② 比较不相并 CJNE (—, —), rel 无并 → 3字节, PC+3
- A, #data
- A, direct
- Rn, #data
- @Ri, data

③ 减1不为0跳 DJNZ (—), rel (循环, 定时)
- Rn
- direct

CJNE记法:
看成减法:
第1操作数 - 第2操作数,
(CY为借位标志位).

跳 { > , CY=0
< , CY=1 }

顺序执行 = , CY=0 顺序执行

外先: rel 如果超过 -128B ~ 127B, 要用跳板.

```
    JZ Loop1
Loop1: LJMP Loop  ⇒ 跳板.
    ⋮ (超过128B)
Loop:
```

空操作 NOP　　PC+1　（也属于改变控制转移类指令）

子程序调用与返回指令

① 绝对调用 ACALL addr11. { PC ← (PC)+2; 不同 LCALL, ← 11位地址 }

同 AJMP（也是11位）
PC为当前值（加二修改后的值）的高5位与指令中的11位地址拼接而成，形成16位的目标地址。

（先压低位，后压高位）

② 长调用 LCALL ← { PC ← (PC)+3; SP ← (SP)+1, (SP) ← PC0~7; SP ← (SP)+1, (SP) ← PC8~15; PC ← 16位地址 }　PCL、PCH 也找

返回：
① 一般返回 RET
② 中断返回 RETI

（先压低位，后压高位）

RET → { PC8~15 ← ((SP)), SP ← (SP)-1; PC0~7 ← ((SP)), SP ← (SP)-1. }

RETI → 除具有RET指令的功能外，还要释放中断逻辑；中断子程序必须用RETI）

为调用和返回指令提供；
为程序的最后一条指令。

---

补充：子程序的独立性与通用性，

编写子程序时要注意：

① 子程序第一条指令必须有标号

② LCALL addr16
　　子标号

③ 最后一条指令必是 RET

④ 注意设置堆栈指针和现场保护

⑤ 子程序可以调用子程序

补充：简答题

1. RET 的作用：自动从堆栈中取出断点地址送给 PC，程序返回断点处继续往下执行。（断点：ACALL 的下一条指令开始往下执行）

2. RETI 的作用：除具有 RET 的功能外，还要释放中断逻辑以接收新的中断请求

位操作类指令 P70~P73

1. 位传送 (2/4个位) 83+128

MOV C, bit

MOV bit, C

注：不能两个bit之间传送，必须经过中间介质C.

2. 位置位

SETB C

SETB bit

3. 位复位

CLR C

CLR bit

注：除了CLR A是对字节其余都是对位

4 位逻辑

与 ANL

或 ORL

非 CPL

} 第1操作数都是C

5. 位条件转移

① 位累加器C状态判断 PC+L

JC rel C=1 跳

JNC rel C≠1 跳

② 位状态判断 PC+3

JB bit, rel (bit)=1 跳

JNB ..同上. (bit)≠1 跳

JBC ..同上 (bit)=1 跳，且bit清0

总结：第一操作数只能是Bit指令

MOV

XCH. XCHD. SWAP

ADD. ADDC. DA. SUBB. MUL. DIV

RR. RL. RRC. RLC

JB. JNB

影响CY. AC. OV位：

ADD. ADDC. SUBB. MUL. DIV

影响CY.

RLC. RRC. CJNE

伪指令 P74~P76.

1. ORG 16位地址

① 规定紧跟在ORG后的源程序的地址

② ORG指示的地址要由小到大排列，不能交叉，也不能重复

2. END

① 整个源程序只有一条END指令，且必须位于程序的最后。

3. EQU (书值) 标识符 EQU 数或汇编符号

① EQU可为程序的任意标号赋值。

② 常量替代

③ EQU要位于主程序的最前面

④ EQU与前面的标号之间不要用冒号，而是用空格分隔

4. DB (定义字节) [标号:] DB 项或项表

① 8位

② 程序存储器中连续单元填充

③ 对DB注意的内容

i) 项与项之间用","分隔；
ii) 字符型数据加""；
iii) 数据可以采用二进制、十六进制以及ASCII.
iv) 负数转换为补码的形式
v) 可多次使用DB定义字节.

DB:
注: ORG 2000H
TAB: DB 12H, 0AFH, 00111001B, "?"

⇒ (2000H) = 12H
(2001H) = AFH
(2002H) = (化为16位)
(2003H) = (化为ASCII码)

5. DW (定义字)

① 16位

② 同DB

DW:
ORG 3000H
TAB: DW 0102H, 7BH, 10
(从3000H开始为高位后存低位)

⇒ (3000H) = 01H
(3001H) = 02H
(3002H) = 00H (空的高位用0填)
(3003H) = 7BH
(3004H) = 00H
(3005H) = 0AH

6. DS 定义存储单元 [标号:] DS 数字

从标号所在地址开始，保留(数字)个字节的存储单元，下一条指令从(标号地址+数字)处开始汇编

7. BIT (位地址符号设置)

标识符 BIT 位地址

# 第四章 中断系统

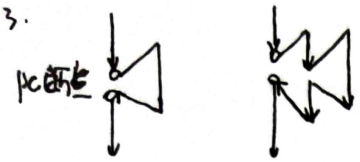
1. 中断源、中断系统

2. 优点：功能、优先级、缺点

↳ 若多个断点

↳ 占用更多堆栈的资源

3.




4. 中断系统图 ☆ 背

中断系统的结构

1、5个中断源
- 2外部中断
  - $\overline{INT0}$
  - $\overline{INT1}$
- 3内部中断
  - T0
  - T1
  - RI和TI

$\overline{INT0}$、$\overline{INT1}$ ← 两种触发方式
- 低电平 — 电平触发
- 下降沿 — 边沿触发

↳
① 检测2次，前1次为高电平，后1次为低电平

② 为保证可靠，低电平和高电平的宽度至少要保持1个机器周期，（12个振荡周期）否则可能会丢失中断事件。

1个中断请求标志.

1. TCON(88H)

| (88H) | TF1 | | TF0 | | IE1 | IT1 | IE0 | IT0 |
|---|---|---|---|---|---|---|---|---|
| | 定时/计数器中断 | | | | 外部中断 | | | |

① IT0(IT1): 外部中断触发方式选择, 软件置1或清0

= 0 时, 电平触发

注: 在该中断服务程序执行完之前, 外部中断源有效低电平必须撤销, 否则将产生另一次中断

= 1 时, 边沿触发 (常用)

② IE0(IE1): 检到外部中断信号后, 硬件置1

(外部中断请求标志)

CPU响应中断后 { 边沿触发: 硬件清0
电平: 软件清0 (需在引脚加触发器)

③ TF0(TF1): 定时/计数器 T0(T1) 溢为中断请求

硬件置1或清0

**编程时注意软件清0的位**

2. SCON(98H)

| (98H) | | | | | TI | RI |
|---|---|---|---|---|---|---|

① RI: 接收中断请求 } 硬件置1, 软件清0

② TI: 发送中断请求

TI和RI经"或"后作为内部的1个中断源

3. IE (中断允许)

| (A8H) | EA | | | ES | ET1 | EX1 | ET0 | EX0 |
|---|---|---|---|---|---|---|---|---|

① EX0(EX1): 外部中断0.允许

② ET0(ET1): 定时/计数器T0溢为中断允许

③ ES 串行口中断允许

④ EA 、(总允许)。

} =1, 允许中断

注: EA=0, 屏蔽所有的断

EA=1, 开放中断, 但每个中断源的中断请求是允许还是禁止, 还需由各自的允许位来确定

4. IP (中断优先级)

| (B8H) | | | | PS | PT1 | PX1 | PT0 | PX0 |
|---|---|---|---|---|---|---|---|---|

① PX0(PX1): 外部中断 ($\overline{INT0}$, $\overline{INT1}$)

② PT0(PT1): 定时/计数器 (T0, T1)

③ PS : 串行口

} =1, 高优先级; =0, 低优先级

2. 基本准则

① 低(优)先级中断可被高 … 打断

高 不可被低 … …

② 同级不能打断同级

3. 自然优先级

高 → 低

$\overline{INT0}$
T0
$\overline{INT1}$
T1
串行口

中断响应过程：（中断响应、中断处理、中断返回）

1、中断响应条件：

① 有中断请求

② EA=1

③ 中断源对应的中断允许位为1

2、CPU在每个机器周期的S5P2时刻采样，检查每个中断源，查询到某个中断标志位为1，且不满足上述在下一个机器周期的S1期间，响应该级最高中断请求

3、阻止条件

① CPU正处理同级或高一级中断

② 当前周期不是执行当前指令的最后一个周期，即要保证把当前的一条执行指令执行完才会响应

③ 当前正在执行返回（RETI）指令或对IE、IP寄存器进行访问的指令时，执行指令后至少再执行一条指令才会响应中断

4、CPU响应中断时完成的工作

① 置位相应优先级状态触发器（即指为中断优先级）

② CPU执行中断服务之前，

i 保护断点：SP ← (PC)

ii PC ← 中断矢量

| | |
|---|---|
| $\overline{INT0}$ | 0003H |
| T0 | 000BH |
| $\overline{INT1}$ | 0013H |
| T1 | 001BH |
| 串行口 | 0023H |

中断处理：

① 中断处理：CPU从执行中断处理程序第一条指令开始到RETI为止

② 包括：保护现场、处理中断源、恢复现场

中断响应时间

| | 机器周期 | |
|---|---|---|
| ① 外部中断标志位到CPU查询到该标志 | 1 | 正常情况 |
| ② 硬件处理子程序调用，中断入口 | 2 | |
| ③ 如果受阻，RETI后跟随乘除指令 | 5 | |

综上，外部中断一般为3-8个机器周期

无优先级：顺序 TCON IP IE

中断响应注意的问题：① 中断程序的响应时间 ② 中断申请信号的宽度 ③ 堆栈的保护

应用实例

1、单步操作中断

2、外部中断源的扩展

① 利用定时器扩展外部中断（性能和硬件要求小）

原理：将T0(T1)设置为模式2外部计数方式（TMOD=06H），并将TL0、TH0都装入#0FFH。

当输入线发生负跳变，TL0+1溢为，TF0被置位（T0中断请求），同时，TH0将0FFH重装入TL0使其恢复，以后对下一次中断。

② 中断加查询方式扩展外部中断源

例：图4-8；P97 编程

中断服务程序最后一条指令必须是中断返回指令RETI

注意：外中断输入均为高电平有效。

外部中断可用电平触发方式

# 第5章 定时/计数器

1. 概述：两个16位定时/计数器，T0和T1。

2. 概念：连续加1计数器（以3脚为准）

对外，计数器 → CPU检测到 ↓ 时，+1，故最高频率为晶振频率的 $\frac{1}{2n}$，这就要求输入信号的电平要在至少一个机器周期内保持不变（即输入信号高/低各1μs，共2μs）↳ 500KHz

对内，定时器 → 每个机器周期 +1（加计数）。（12分频，即1μs）再接收一个计数脉冲的时间间隔为1μs

3. 存放初值的SFR：（赋值时要分开赋值）

T0 TH0, TL0

T1 TH1, TL1

4. TMOD, 和 TCON ① 控制/管理定时/计数器两个

② 返回当前工作等在器件下状态

6. 各方式定时最大时长（6MHz）

方式0：16.384 ms

1：131.072 ms

2：512 μs

4. 初值计算：

最大值M，计数值N，初值X，则

计数初值 $X = M - N$

定时初值 $X = M -$ 定时时间 $/ T_{机器周期}$ （方式0计算初值时需要转化!!）

（$T = 12/$晶体振荡频率；1μs）

5. ~~P1口系统把用~~ 背 I II

两个控制位

1. TMOD 工作模式

T1模式 T0模式

| (89H) | GATE | C/T | M1 | M0 | GATE | C/T | M1 | M0 |
|---|---|---|---|---|---|---|---|---|

不能位寻址

① M1, M0：工作模式选择

0 0 13位，TH 8位和 TL 低5位

0 1 16位

1 0 自动重装8位

1 1 T0分成两个独立8位，T1无模式

② C/T 方式选择

=1 计数

=0 定时

③ GATE 门控位

GATE=1，TR1=1 且 $\overline{INT1}$=1：允许T1计数 → 测量 $\overline{INT0}$（或 $\overline{INT1}$）端出现的正脉冲宽度

GATE=0 TR1=1：允许T1计数

TR0=0：禁止……

2. TCON 定时器控制

T1 T0 中断（与本章无关）

| (88H) | TF1 | TR1 | TF0 | TR0 | IE1 | IT1 | IE0 | IT0 |
|---|---|---|---|---|---|---|---|---|

① TF0（TF1）：T0（T1）溢出标志位

加1计满溢出时 硬件置1，并申请中断。

硬件清0

② TR0（TR1）：T0（T1）运行控制位

软件置1，开始计数

复位时清0

相当定时/计数器的开关

=1 开

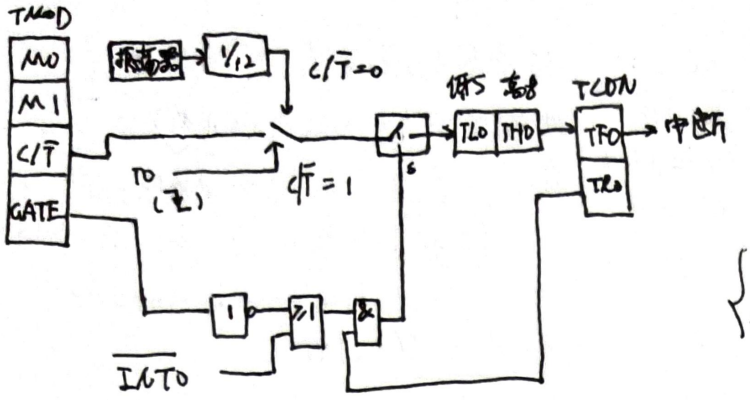
=0 关，想关就关

P3.2 P3.3

四种工作方式.

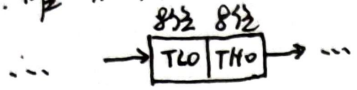
方式 0.

1. 逻辑框图

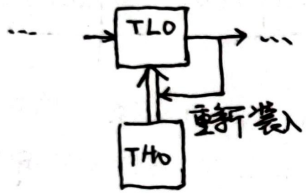



方式 1

1. 逻辑框图 同上. 唯一区别是




方式 2.

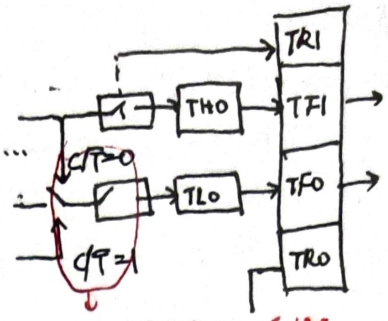



特别适用于 串行口波特率发生器(可产生相当精确的定时时间)

TH0用于保存初值.

赋值时. TL0 和 TH0 都要赋值, 且赋的值一样.

方式 3.




C/T控制 TL0 功能

① TL0 为原 T0 功能

② TH0 只能定时.

占用 T1 的 TR1 和 TF1, 和中断入口地址(001BH)

启动关闭只受 TR1 控制

∴此时 T1 只能用于不需要中断的场合.

③ T1 无方式3.

T0 方式3时. T1 仍可方式0或2

☆一般情况下, T1 用做串行口波特率发生器时, T0 才做方式3.

T1 用于不用中断的场合, 只能将输出送入串行.

此时 T1 系统自动:

T1 仅有 C/T 位可用

定时/计数器实例

1. 初始化步骤.

① TMOD

② TH, TL.

③ IE

④ IP (也可以用查询方式)

⑤ TCON

2. 注意:

① 当使用 T1 不用 T0 时, T0 设成方式3以防止其他动作

② 对于没有用到的寄存器全部写0.

③ 查询 → 要及时对 TF 清0 中断

④ 不在方式2时, 要及时重新写初值 (在中断服务程序中重新赋值)

外部利用定时计数器产生脉冲 → C/T=0

(产生方波(波特率) → 定时器)

用于 扩展外部中断. → C/T=1

(注: 利用查询方式就不用管IE寄存器了)

② 利用查询方式实现定时/计数功能

```
CHK: JBC  TF0, ZR    ;当TF0=1时跳
     STJMP CHK       ;持中断  并清0
ZR: ...
```

③ 循环中断的功能:

```
恢复定时器初值后,
DJNZ R4, LOP
...
LOP: RETI
```

第六章、串行口　并行通信：数据以各位同时进行传送
　　串　　：数据一位一位地顺序进行传送

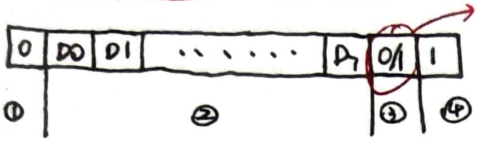
串行通信
- 并行通信
- 串行
  - 串行同步通信
  - 串行异步通信
    - 单工
    - 半双工
    - 全双工

（两机通信只有在波特率相同的情况下才能进行传输）

## 6.1

### 1. 异步

① 以字符（字节）为传输组成数据帧




可编程为第9位

还不懂？

② 起始位：始终为低电平0。

③ 数据位：可发送5～8位数据，ASCII常为7位；由低位到高位依次传送

④ 奇偶校验位

⑤ 停止位：始终为高电平1

帧间：可能有空闲位，在第n个停止位之后，第n+1个起始位之前，始终为1（RXD由1→0，启动接收）

② 场合：数据量不大、速率不高、远距离

### 2. 同步

有起始位、停止位，传输速率要求不需要发送同步的时钟，字符帧长度不固定。

① 同步字符：采用一个或两个同步字符作为起始位，接收端不断采样并比较，同步字符相同后才能传送。

② 场合：数据量大、速率高的场合

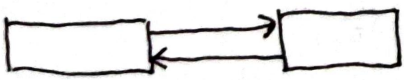
## 6.2 串行通信的制式

### 1. 单工

### 2. 半双工

（同一时刻只能接收或发送）

### 3. 全双工



## 6.3 波特率

### 1. 概念

串行通信的数据是按位传输的，

每秒传送的二进制的位数称为波特率。

单位：bit/s

每位的传送时间为波特率的倒数

例：110 bit/s

$T_d = \frac{1}{110} \approx 0.0091\ s = 9.1\ ms$

发送/接收时钟

① 发送器在发送时钟的下降沿发送；接收器上升沿采样

② 发送/接收频率 = n × 波特率

n为波特率因子=1或16或64

同步必须　异步（通常）

每1位传输时间 $T_d = nT_c$

$T_c$：发/收时钟周期

奇偶校验

## 6.4 串行口的结构

1. 引脚 RXD（P3.0），TXD（P3.1）
2. 系统图 P117. 图6-5
3. 两个物理上相对独立的数据缓冲器 SBUF。

① 发送缓冲器（SBUF）只能由CPU写入要发送的数据　不能读出　MOV SBUF, A

② 接收 SBUF，只能由CPU读出，不能写入　MOV A, SBUF

双缓冲前后2帧数据重叠，接收器是双缓冲。

共用 99H 的地址，

通过不同的指令来识别/发送还是接收。

寄存器

1. SCON

| (98H) | SM0 | SM1 | SM2 | REN | TB8 | RB8 | TI | RI |
|---|---|---|---|---|---|---|---|---|

① RI：① 方式0 收到第8位结束或其他方式收到停止位

② 表示收到了一帧数据

③ 收到有效数据才会置1

④ 由硬件置位，软件清0。

② TI：① … 发送 … 发送…

② 表示可以发送第二帧数据

③ 硬件置位，软件清0。

③ RB8：① 方式2、3 中接收第9位并放入RB8

② 方式1，SM2=0，则RB8 接收到停止位

③ 方式0，该位不用

④ TB8 ① 2、3 … 发送 … ，可由软件定义，常作奇偶校验 MOV C, PSW.0 MOV TB8, C.

⑤ REN（接收允许位），=1 允许接收

=0 禁止接收

软件置位或清0。（允许接收位）

⑥ SM2：① 方式2、3 用于多机通信（多机通信控制位）

=1 只有第9位数据RB8=1 才置RI

=0 不论 … RB8是否为1，都置位，将收到的数据装入SBUF

② 方式1，（RB8=1）

=1 只有当接收到的停止位为1时才置位RI

③ 方式0，必须始终为0

2. PCON 87H 不可位寻址

① 只有1位 SMOD 与串行口工作有关

② SMOD=1 时，波特率加倍

用于方式1、2、3 的波特率

补充：

① SCON中，

方式0 对于 SM2、TB8、RB8 都不用（=0）

方式1 对于 TB8 不用

对于 RB8 存放停止位

② 弄清 SM2、TB8、RB8 的关系

方式0：不用

方式1、2、3：RB8 存放第9位，（对于1位的第9位是停止位）同时

SM2=1，RB8=1 ⇒ RI=1

SM2=0，X ⇒ RI=1

总结：目前学习过的专用寄存器

| | | |
|---|---|---|
| PCON | 87H ✓ | 不可位 |
| TCON | 88H | |
| TMOD | 89H ✓ | |
| SCON | 98H | |
| SBUF | 99H ✓ | |
| IE | 可A8H | |
| IP | B8H | |

7.

| SM0 | SM1 | 4种串行口工作方式 | 波特率 | 主要用途 |
|---|---|---|---|---|
| 0 | 0 | 同步移位寄存器 | $f_{osc}/12$ | 扩展并行口 |
| 0 | 1 | 10位异步收发 | 可变 | 串行口通信 |
| 1 | 0 | 11位异步收发 | $f_{osc}/32$ 或 $f_{osc}/64$ | 串行口通信 |
| 1 | 1 | 11位异步收发 | 可变 | 串行口通信 |

四种工作方式

1. 方式0 —— 并接串入并出，或串入并出，作为总体

特征：① 同步移位，用于扩展并行I/O

② RXD 数据（输入输出）
TXD 时钟（输出同步移位脉冲）

③ 8位，无起始、奇偶校验、停止

| （1帧数据8位数） | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| … | D0 | P1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 | … |

① 发送
① 数据 → SBUF 数据 → RXD 数据 →
② 数据发送完 硬件置位 TI
发送下一次数据前 软件清0 TI

② 接收
① 用软件使 REN=1，RI=0，启动一次接收
② 数据 → RXD 数据 → SBUF → A
③ 接收完数据 硬件置位 RI
通过中断或查询方式将数据读入累加器
接收下一数据前 软件清0 RI

2. 方式1

10位

| 起始 | D0 | D1 | … | … | D7 | 停止 |
|---|---|---|---|---|---|---|

特征：① 波特率可变的10位异步通信
② TXD 发送
RXD 接收
③ 停止位 → RB8

① 发送 ① 从低位到高位逐位发送

② 接收
① 软件 REN=1 后，接收器以所选波特率的16倍的采样脉冲对 RXD 采样，当 RXD 由1→0，接收数据
② 接收完数据后：
RI=0，SM2=0 或 接收到的停止位为1 → { SM2=0，不管 RB8，只需 RI=1 ; SM2=1，且有 RB8=1 }
则 接收有效不丢失
（8位数据 → SBUF，停止位 → RB8，置位 RI）
读出 ← CPU中断 ←
否则 数据将被下一帧数据覆盖

方式2，方式3

特征：① 都是11位异步，操作方式一样
② 只有波特率不同
③ 11位

| 起始 | D0 | D1 | … … | D7 | D8 | 停止 |
|---|---|---|---|---|---|---|

④ 奇偶校验或地址/数据选择
⑤ 发送时为 TB8
接收 RB8

① 发送 !! 注意顺序
① 软件设置 TB8 ② 数据写入 SBUF
发送完硬件置位 TI
∵ 数据写入 SBUF 就相当于发送
∴ 要先予TB8后化

② 接收 接收允许位
① 软件 REN=1，采样同方式1
② 当接收到第9位数据送入接收移位寄存器后
若 RI=0 且 SM2=0
或 接收到的第9位数据为1（SM2=1）
则有效不丢失，第8位数据 → SBUF
第9位 → RB8，
硬件置位 RI，
否则 … 丢弃

# 波特率设置

方式0：波特率 = $f_{osc}/12$

方式2：$= \frac{2^{SMOD}}{64} \times f_{osc}$ （SMOD=0或1）

方式1、方式3：

$= \frac{2^{SMOD}}{32} \times$ 定时器T1溢出率

$= \frac{2^{SMOD} \times f_{osc}}{32 \times 12 (2^n - X)}$

T1定时时间决定波特率

其中，定时器时间 $T_C = (2^n - X) \times T_{机器周期}$

$T_{机器周期} = \frac{12}{f_{osc}}$

$\therefore$ T1溢出率 $= \frac{1}{T_C} = \frac{f_{osc}}{12(2^n - X)}$

此时，T1作波特率发生器使用，可工作于模式0、1、2 ✓ 常用

波特率的相对误差 ≤ 2.5%

① 先用 SMOD=1 算出X值，再将此X代入求得实际值

最后 (实际值 − 理论值)/理论值 = 相对误差

② 再用 SMOD=0 按同样的步骤求得相对误差

取相对误差 ≤ 2.5% 的值

# 串行口的应用 （省程序不用改波特率）

1. 步骤 ① 设定一致的波特率（T1）
② 设定一致的串行口工作方式
③ 分别设计中断服务程序或查询程序
注意对T1、对方处理（硬件置位、软件清0）
④ 各自发送、接收程序可以不同

## 1. 串行口方式0的应用

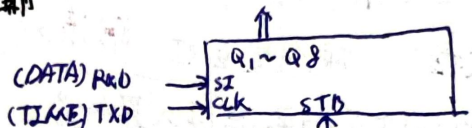
- ① 利用串行口扩展并行输出口

外接串入一并出芯片 CD4094

- ② 利用 …… 输入口

外接并入一串出芯片 CD4014

其中：CD4094，扩展输出口 → 第八一条件 控制端=1 允许输出



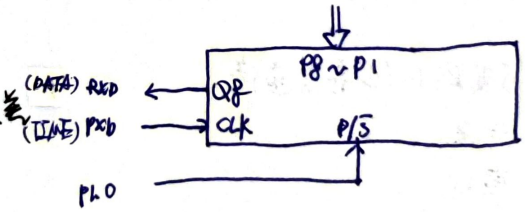

（可能7题）

P1.0 相当于方法，如果有2片 CD4094，用P1.1连第2片

① STB=0，则 输出口(Q1~Q8)关闭，允许SI输入

② STB=1，则 输入寄存器(SI)关闭，允许Q1~Q8输出

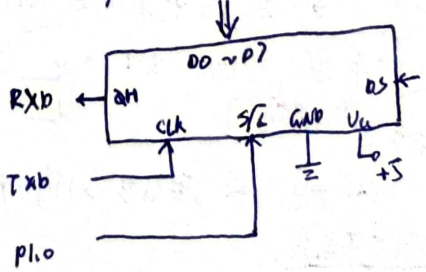
CD4014 扩展输入口 → 第八一条件 控制端=1 允许输入




① P/S=0，输入寄存器P8~P1关闭，允许Q8输出

② P/S=1，为Q8关闭，允许P8~P1入

74HC165 扩展输入口




① S/L=0，并行输入

=1，输出

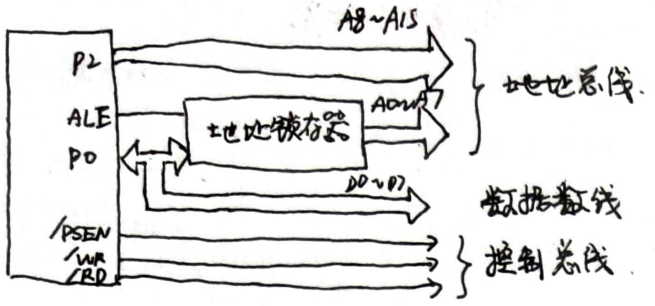
补充：判断奇偶校验位模板

发送：MOV C, PSW.0
MOV TB8, C

接收：JNB PSW.0, PZEO
JNB RB8, ERR
SJMP RIGHT
PZEO: JB RB8, ERR

第七章. 扩展.

1. 单片机三总线结构




① 地址总线 (AB)
- ① 16位，可寻址范围 $2^{16}B = 64KB$
- ② 单向.

② 数据总线 (DB)
- ① 8位 → P0口，地址/数据复用线.

③ 控制总线 (CB)
- ① ALE：(锁存P0口输出的低8位地址)的选通信号
- ② $\overline{PSEN}$：扩展程序存储器读选通信号
- ③ $\overline{EA}$：片内或片外程序存储器的选择信号
  - ⇒ 只访问外部程序存储器 → $\overline{EA}$ 接地
- ④ $\overline{RD}$，$\overline{WR}$：片外数据存储器和扩展I/O口的读写选通信号
  - ⊖ MOVX 时，自动有效.

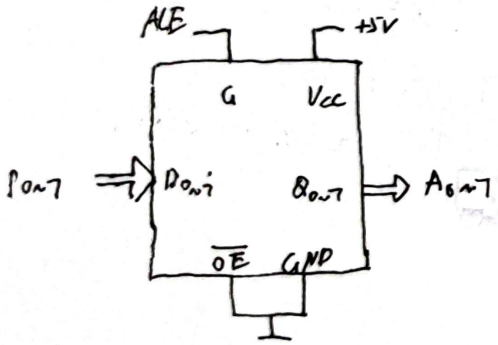
扩展系统编址方法

1. 线选法 P164 例1 7-8. } 都要编地址!!
2. 译码法 P165 例1 7-9.
   - 应用 74LS138

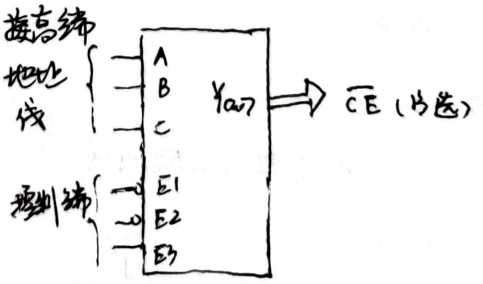
---

2. 应当了解的芯片

① 74LS373 (下跳沿锁存)




② 74LS138 译码器.




i) 接线方法：
E1、E2 都接低电平，E3 接高电平，则 74138 为监听状态

ii) 工作方式

| C | B | A | $Y_0$ | $Y_1$ | … | $Y_7$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | … | 1 |
| 0 | 0 | 1 | 1 | 0 | … | 1 |
| … | | | … | | | |
| 1 | 1 | 1 | 1 | 1 | … | 0 |

即 $Y_i = 0$ 为通

2. 程序存储器的扩展.

问作用：程序存储器 or 数据～ 作用：
问是什么：只读～ or 随机～

1. EPROM（紫外线擦除的可编程只读存储器）

① 型号 2764（8kB×8），例 27128（16K×8）

② 引脚.

作用：做程序存储器使用

① A0~Ai（i=10~15），2764 为 12

② D0~D7

③ $\overline{CE}$ 片选 —— A15

④ $\overline{PGM}$.

⑤ $\overline{OE}$ 读选通 —— $\overline{PSEN}$

⑥ $V_{PP}$

⑦ $V_{CC}$、GND

③ 工作方式

① 读出：$\overline{CE}=0$

输入 A0~A12

$\overline{OE}=0$

D0~D7 得数据

时序

同时，$\overline{PGM}$ 接高电平；接线 $\overline{PGM}$、$V_{PP}$ → +5V

② 维持：$\overline{CE}$ 无效，即芯片没有被选中，

数据总线处于高阻态，

④ 编程

⑤ 程序校验

⑥ 禁止编程

2. $E^2$PROM（电擦除可编程存储器）

① 型号 2817（2kB×8）

② 引脚. P164

RDY/$\overline{BUSY}$，2817的空/忙状态输出线

③ 工作方式

1) 读出

2) 写入

3) 维持.

时序图 P148.

扩展接线方式，P148. 一般方法

P149. 例 7-1、7-2、7-3

（2732）（2764）（2817A）

3. 数据存储器的扩展.

① 型号 6264 8K

是：随机存储器

② 引脚：

① A0~Ai，6264：12.（地址输入）

② D0~D7，双向三态数据线

③ $\overline{CE}$，NC；

NC 为高，$\overline{CE}$ 为低，} 选中芯片

④ $\overline{RD}$，$\overline{WR}$.

⑤ $V_{CC}$，GND

③ 工作方式：读、写、维持

时序：1. 片选
2. 输入地址
3. $\overline{RD}$

1. 片选
2. 输入地址
3. 把要写的数据放入 D0~D7
4. $\overline{WR}$.

④ 时序图.

⑤ 接线方式 P153 一般方法

P153. 例 7-4. 7-5

6116 2864.

补充：时序图需要注意的地方

① 第一个机器周期是从外部程序存储器读取 MOVX 指令操作码，

第二个机器周期才是执行 MOVX 访问外部数据存储器

② 只要不执行 MOVX，ALE 信号总是每个机器周期 2 次有效，$f_{ALE}=f_{osc}/6$.

∴ ALE 可作为外部时钟信号

但在 MOVX 指令周期中，ALE 只有一次有效，

$\overline{PSEN}$ 始终无效

对地址编址时，取并使用作为

注意：设计地址范围时不要和 51 内的程序存储器地址重叠

4. I/O口的扩展.

4.1 简单I/O口的扩展 (D0~D7 输入, Q0~Q7 输出)

1. 并行输出口.

① 74LS273 (8D触发器, 上升沿触发, 具有锁存)

CLK.

无片选 → 或门选通利用

P155 图7-15. 用74LS273扩展输出口 省及地址

CLK

② 74LS377 (8D触发器, 上升沿触发, 有锁存)

有片选 $\overline{EN}$

P156. 图7-17. 用74LS377扩展输出口.

2. 并行输入口.

① 74LS244 (三态, 输出, 无锁存)

只有三态输出的芯片才能挂在CPU上作输入口

引脚: 1A1~1A4, 2A1~2A4 } 输入
1Y1~1Y4, 2Y1~2Y4 } 输出
$\overline{1G}$, $\overline{2G}$ } 控制端 { 控制1组选通, 控制2组选通 } 相当于门选

连接方式:

P156. 图7-18. 用74LS244扩展输入口 省及地址

② 74LS245

有方向.

连接方式P157. 图7-21

4.2 8155可编程多功能接口的扩展

① 结构

1. 256×8bit, 静态RAM.
2. I/O口
   PA, PB: 可编程8位
   PC: ... 6位
3. 14位二进制减1可编程定时计数器.

8051: +1定时法

② 引脚

AD0~AD7 (可与P0直接相连)
PA0~PA7, PB0~PB7, PC0~PC6
$\overline{CE}$
ALE
$\overline{RD}$, $\overline{WR}$
Vcc, GND (上面多常见引脚)
IO/$\overline{M}$: IO口在RAM选择信号输入线.
TIMER IN (TIN): 定/计的计数脉冲输入线
TIMER OUT (TOUT):
RESET: 高电平有效, 脉冲典型宽度为600ns, 8155被复位后, 三个I/O被置为输入方式

③ 8155 RAM与I/O的编址及寻址

i) IO/$\overline{M}$=0, RAM读/写, 地址为00H~FFH (256B)

ii) IO/$\overline{M}$=1, I/O读写, 地址如下

① 命令口和状态口使用同一个地址;
② 写是命令口, 读为是状态口

| A7~A3 | A2 | A1 | A0 | I/O命令口/状态口 |
|---|---|---|---|---|
| X ... X | 0 | 0 | 0 | |
| X ... X | 0 | 0 | 1 | PA |
| X ... X | 0 | 1 | 0 | PB |
| X ... X | 0 | 1 | 1 | PC |
| X ... X | 1 | 0 | 0 | TL |
| X ... X | 1 | 0 | 1 | TH |

④ 8155工作方式

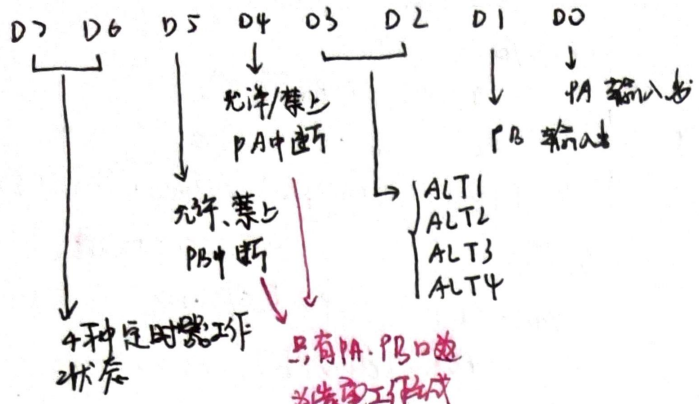
注: PA, PB都有以下两种工作方式.
PC只有第1种, 但也能为工作在第1种方式的PA、PB提供控制线

i) 基本输入/输出方式 (P160. 图7-26)

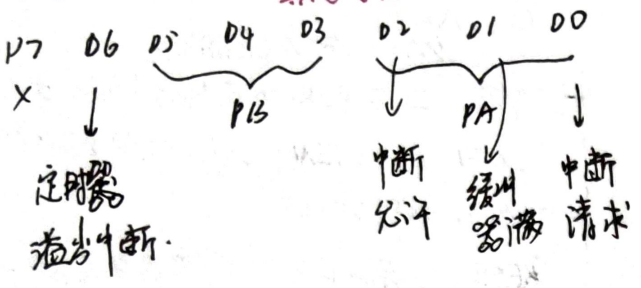
ii) 选通输入/输出方式 (P160. 图7-27)

⑤. 8155 命令字，状态字.

i) 命令字（写） （具体于P158.）




ii) 状态字. (读)




⑦ 8155 与 C51 的连接方式

P162.

① 连接方式

② 地址

~~③ 引脚~~

RAM : 7E00H ~ 7EFFH

P2.0 为 IO/M 填为0

P2.7 为 CE，填为0

命令.状态口: 7F00H

PA 7F01H

PB 7F02H

PC 7F03H

定时器低8位 7F04H

定时器高8位 7F05H

④例题外有知识点：

对输入方波进行分频，即对输入脉冲计数。

前12个脉冲输入，则输出1个高电平；后12个……，依此。

24个输入脉冲，芯片输出1脉冲。

→正常工作要设置 ①工作状态、②定时器常数（即初值）③ TOUT 输出信号形式

① 8155定时/计数器（14位减1）

1. 工作状态中命令字高两位决定.（4种）

00：空操作

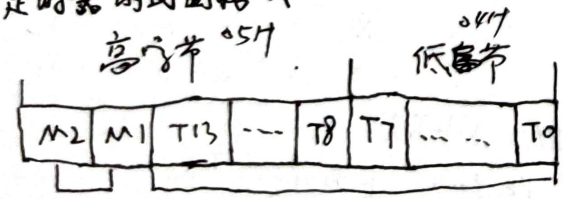
01：停止工作

10：①未启动 则空操作，②启动，则计数到0时停止工作。

11：①启动，并在设置完初值于输出允许后开始工作

②工在工作，则计数到0后，以新设初值和输出方式工作。

2. 定时器的时间格式




| | | |
|---|---|---|
| 0 | 0 | 单个方波 |
| 0 | 1 | 连续方波 |
| 1 | 0 | 单个脉冲 |
| 1 | 1 | 连续脉冲 |

①自动恢复初值
②时间常数最好为偶：输出方波对称
奇：不对称→高电平比低电平多一个计数间隔

③ X最最小为2 ∴ X范围：0002H ~ 3FFFH

④ TIN 输入脉冲的最高频率为4MHz

外先：

①TIN 输入计数脉冲，当计数器减到0时，从TOUT 输出一个信号，同时将状态字中的 TIMER 置位（读出后清0）

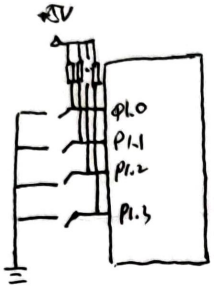
②使用时须先设定其工作状态、计数初值及TOUT引脚输出信号形式

第14位和15位为1，装不到14、15位

图 P161.

第八章. 单片机的I/O接口技术.

1. 键盘.
   - 1、分类：编码键盘，非编码键盘.
   - 2、工作原理.
     - ① 抖动现象.
       1、5~10 ms
       2、解决方法：软件延时.
       (第一次检测到键闭合先不做相应动作，而是执行一个延时程序，产生5-10ms的延时，让前沿抖动消失后再次检测键的状态，若仍保持闭合状态，则确认为真正有键按下.)
     - ② 速度的处理
   - 3. 键盘结构及其程序
     - ① 独立式非编码键盘

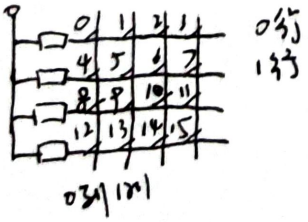



       i) 特点：
       ① 每一个键对应P1口的一根口线
       ② 没键对应的口线的电位由高转变为低时，cpu访问并查询所有口线
       ii) 程序. (P168. 下面) 背
     - ② 矩阵式.
       i) N条行线，m条列线，共N×M个键.
       ii) 识别方法：扫描法：

老师讲的：行作输入，列输出，行=0，列按下

① 行线逐行发0，读列线状态，(列)为1.没按 (列)=0，按下
② 延时除抖动
③ 判断键号
④ 检查按键已释放

       iii) 编址.（矩阵键码=该行首键码+该列号）

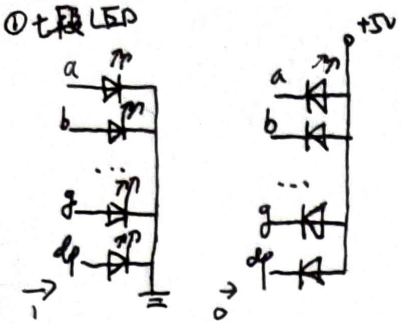



       iv) P1.0 程序设计

2、显示器及接口电路.

1. LED数码显示.

① 七段LED




共阴 共阳 普脚分布.

加高脚点亮 加低脚点亮. 两个com为公共端.

(一般选择共阴) 共阴接地

共阳接 $V_{CC}$.

② 段码

| 代码位 | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|
| 显示段 | dp | g | f | e | d | c | b | a |

排阻与排线有P176，表8-1 段码.

③ 接口及显示方式

i) 静态显示

数码管的每一段均连与一个8位锁存器的输出相连. 显示字符一经确定，相应锁存器的输出将维持不变.

N位显示，需 N×8 根输出控制线.

P175 图8-8.

ii) 动态显示

1. 概念 P176 上方

2. 工作方式：扫描的方法

3. 程序设计时应延时子程序

取每位LED显示的时间间隔不超过20ms，点亮时间为1~2ms

八、A-D、D-A 转换器与单片机接口（相对外部RAM）

1. D-A 转换器

① 原理：送到 DAC 的各位二进制数按其权的大小转换为相应的模拟分量，再把各模拟分量叠加，其和就是 D/A 转换的结果。

$$V_o = -V_{REF}\sum_{i=0}^{n}\frac{D_i}{2^n}$$

② 主要参数

i) 分辨率：D-A 转换器能够转换的二进制位数（输出满刻度时的模拟量与 $2^n$ 之比）

ii) 转换时间

iii) 线性度

iv) 输出电平

③ DAC0832

两个缓冲器 { 单缓冲方式 √（1号）
双 ---

① 转换速度 1μs
② 可直接与主机相连

数据总线 DI0~DI7

输入数据寄存器(8位) → DAC 寄存器(8位) → D-A 转换器(8位)

VREF 基准(参考)电压
IOUT1 / IOUT2 电流输出
Rfb 反馈信号输入端

锁存允许，常直接接电源

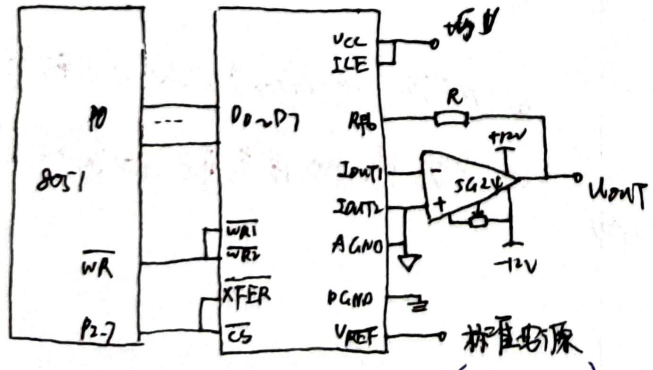
ILE=1
$\overline{CS}$=0, $\overline{WR1}$=0 数据信号被锁存

ILE
$\overline{CS}$
$\overline{WR1}$
$\overline{WR2}$
XFER
&

AGND 模拟地
VCC
DGND 数字地

数据传送信号

DAC 0832 工作于单极性单缓冲方式线路图




基准电压源

$V_{REF}$ 接 +10V 或 -10V 时
$V_{OUT}$ = 0~+10V 或 0~-10V

$V_{REF}$ 接 +5V 或 -5V 时
$V_{OUT}$ = 0~+5V 或 0~-5V，即 $V_{OUT} \in (0, X)$

① 输出电压与参考电压为相反数。

② DAC0832 是电流型，所以要外加运放将 I 转化成 V。

P187. 锯齿波程序

深 P189 页笔记：锯齿波 三角波 矩形波

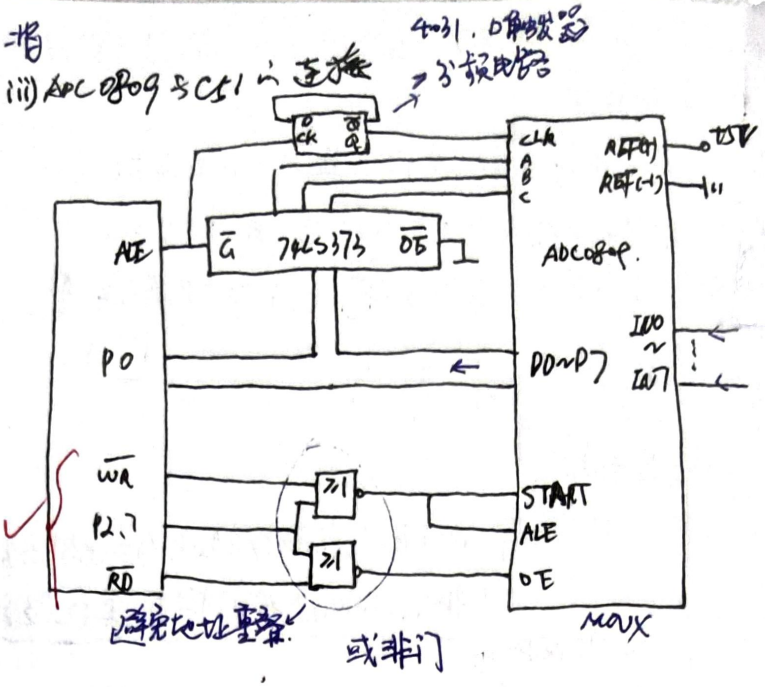
# A/D 转换器

- ① 原理：逐次逼近式，P187 原理图
- ② 主要参数
  - i) 分辨率
  - ii) 量程
  - iii) 精度
  - iv) 转换时间
- ③ ADC0809（分辨率为8位）
  - i) P187 结构框图
  - ii) 引脚：
    - ① IN0～IN7：8路模拟量输入通道
    - ② 单极性，0～5V
    - ③ 虽有8个输入通道，但同一时刻只能对1个模拟量进行转换
    - ④ A、B、C：控制 IN0～IN7 的选通（7FF8H～7FFFH）
    - ⑤ START = 启动（上升沿：清除A中内部寄存器；下降沿：启动）
    - ⑥ ALE：上升沿锁存A、B、C（系统通过软件输入一个正脉冲，启动A-D转换）
    - ⑦ EOC：转换结束状态信号，=1，通知CPU转换结束
    - ⑧ D0～D7：输出
    - ⑨ OE：输出允许，=1，输出转换后的8位数据
    - ⑩ CLK：ADC0809没有时钟电路。当系统主频为6MHz时，ALE为1MHz，ADC0809的CLK为500kHz（延时128μs）

$$f_{ck} = f_{ALE} = \frac{1}{6} f_{osc}$$

$$f_{CLK} = f_Q = \frac{1}{2} f_{ck}$$

iii) ADC0809与CPU的连接




P2.7 置为低电平，IN0～7 的地址为 7FF8H～7FFFH 各选一

P0.0～P0.2（低3位）：用于切换通道 A B C

iv) 时序 P188

v) 编程时要注意

- ① 启动 AD 接口设计
- ② 转换时钟的提供，CPU 的晶振 6MHz
- ③ 转换完成的判断（EOC 从0到1，若用延时的方法就不用管）
- ④ 模拟通道的选择

vi) 编程举例

- ① P189 延时输入方式
- ② P190 查询、中断方式

① MOVX @DPTR, A；启动 A-D 转换

② 8路模拟信号轮流采样程序